Exam. Seat No. | |

Walwa Taluka Education Society's

YASHWANTRAO CHAVAN MAHAVIDYALAYA, ISLAMPUR

Tal: Walwa, Dist: Sangli.

Department Of Computer Science

LABORATORY CERTIFICATE

This is to certify that, Mr./Miss.

_____

Roll No. _____ has satisfactorily completed the course of Experiment in **Major Elective Practical I - Core Java Lab** as per syllabus laid down by Shivaji University for **B.Sc. Computer Science Part –III Sem V** during the academic year 2026-2027.

Date: / / 2026

Teacher in charge **Examiner** **Head**
Department of Computer Science

INDEX
Major Elective Practical I - Core Java Lab

Sr. No.	Title	Page No.	Date	Remark
1.	Write a Java program to demonstrate all primitive data types and print their values.			
2.	Write a program to input 3 numbers on command line argument and find maximum number of them.			
3.	Write a Java program to demonstrate branching statements (if,if-else,switch)			
4.	Write a Java program to demonstrate lopping statements (for,while,do).			
5.	Write a Java program to demonstrate implicit and explicit type casting.			
6.	Write a Java program to demonstrate constructor and this keyword.			
7.	Write a Java program to demonstrate static variable and static method.			
8.	Write a Java program to demonstrate inheritance.			
9.	Write a Java program to demonstrate method overloading and methog overriding.			
10.	Write a java program to create an interface shape with the getArea() method. Create three classes Rectangle,Circle,and Triangle that implements the shape interface. Implement the getArea() method for each of the three classes.			
11.	Write a Java Program to demonstrate Abstract class and abstract method.			
12.	Create a package College and add two classes Student and Staff to it.Add appropriate members to the classes.			
13.	Write a Java program that throws an exception and catch it using a try-catch block.			
14.	Write a Java program to create a thread using: .Extending Thread class .Implementing Runnable interface			

15.	Write an Awt program to demonstrate FlowLayout, GridLayout, BorderLayout and CardLayout.			
16.	Design a simple GUI with a Label and Button. Each time user clicks the button increase counter and display number of counts on the label.			
17.	Develop a GUI application using AWT to create a simple calculator using: . Label . TextField . Button . Layout Managers			
18.	Design a simple GUI Temperature converter, using appropriate component.			
19.	Write a java program using swing to create a frame having three text fields. Accept number in first textfield and display previous number in second textfield and next number in the third textfield.			
20.	Write a Java program that works as a simple calculator. Use a grid layout to arrange buttons for the digits and for the +, -, *, % operations. Add a text field to display the result. Handle any possible exceptions like divide by zero.			

Practical No: 1

Program: Write a Java program to demonstrate all primitive data types and print their values.

Program Analysis: The program demonstrates all eight primitive data types in Java by declaring variables and assigning values to them.
It then prints the values of each data type using System.out.println().

Source Code:

```
class PrimitiveDataTypes
{
   public static void main(String[] args) {
      byte b = 10;
      short s = 1000;
      int i = 50000;
      long l = 1000000000L;
      float f = 10.5f;
      double d = 20.1234;
      char c = 'A';
      boolean bool = true;
      System.out.println("Byte: " + b);
      System.out.println("Short: " + s);
      System.out.println("Int: " + i);
      System.out.println("Long: " + l);
      System.out.println("Float: " + f);
      System.out.println("Double: " + d);
      System.out.println("Char: " + c);
      System.out.println("Boolean: " + bool);
   }
}
```

OUTPUT:

```
Byte: 10
Short: 1000
Int: 50000
Long: 1000000000
Float: 10.5
Double: 20.1234
Char: A
Boolean: true
```

Practical No: 2

Program: Write a program to input 3 numbers on command line argument and find maximum number of them.

Program Analysis: The program accepts three numbers from the command line arguments and converts them from String to int.
It compares the three numbers using if-else statements and displays the maximum number.

Source Code:

```
class MaximumNumber {
   public static void main(String[] args) {
      int a = Integer.parseInt(args[0]);
      int b = Integer.parseInt(args[1]);
      int c = Integer.parseInt(args[2]);

      int max;

      if (a >= b && a >= c)
         max = a;
      else if (b >= a && b >= c)
         max = b;
      else
         max = c;

      System.out.println("Maximum number = " + max);
   }
}
```

OUTPUT:

```
java MaximumNumber 25 47 32
Maximum number = 47
```

Practical No: 3

Program: Write a Java program to demonstrate branching statements (if,if-else,switch).

Program Analysis: The program demonstrates decision-making statements in Java: if, if-else, and switch.
It checks conditions and displays the appropriate result based on the given values.

Source Code:

```
class DecisionStatements
{
   public static void main(String[] args)
 {
       int number = 10;
       int marks = 75;
       int choice = 2;
       // if statement
       if (number > 0)
          System.out.println("Number is positive");

       // if-else statement
       if (marks >= 40)
          System.out.println("Student is Pass");
       else
          System.out.println("Student is Fail");

       // switch statement
       switch (choice) {
          case 1:
             System.out.println("You selected One");
             break;
          case 2:
             System.out.println("You selected Two");
             break;
          case 3:
             System.out.println("You selected Three");
             break;
          default:
```

```
            System.out.println("Invalid choice");
        }
    }
}
```

OUTPUT:

Number is positive
Student is Pass
You selected Two

Practical No:4

Program: Write a Java program to demonstrate lopping statements (for,while,do).

Program Analysis: The program demonstrates the three looping statements in Java: for, while, and do-while.
Each loop is used to print numbers from 1 to 5.

Source Code:

```
class LoopingStatements {
   public static void main(String[] args) {

      // for loop
      System.out.println("For Loop:");
      for (int i = 1; i <= 5; i++) {
         System.out.println(i);
      }

      // while loop
      System.out.println("While Loop:");
      int j = 1;
      while (j <= 5) {
         System.out.println(j);
         j++;
      }

      // do-while loop
      System.out.println("Do-While Loop:");
      int k = 1;
      do {
         System.out.println(k);
         k++;
      } while (k <= 5);
   }
}
```

OUTPUT:

For Loop:
1
2
3
4
5

While Loop:
1
2
3
4
5

Do-While Loop:
1
2
3
4
5

Practical No: 5

Program: Write a Java program to demonstrate implicit and explicit type casting.

Program Analysis: The program demonstrates **implicit type casting**, where Java automatically converts a smaller data type into a larger data type.
It also demonstrates **explicit type casting**, where the programmer manually converts a larger data type into a smaller data type using a cast operator.

Source Code:

```
class TypeCasting {
   public static void main(String[] args) {

      // Implicit type casting
      int num = 100;
      double d = num;

      System.out.println("Implicit Casting:");
      System.out.println("Integer value = " + num);
      System.out.println("Double value = " + d);

      // Explicit type casting
      double value = 25.75;
      int n = (int) value;
      System.out.println("\nExplicit Casting:");
      System.out.println("Double value = " + value);
      System.out.println("Integer value = " + n);
   }
}
```

OUTPUT:

```
Implicit Casting:
Integer value = 100
Double value = 100.0

Explicit Casting:
Double value = 25.75
Integer value = 25
```

Practical No: 6

Program: Write a Java program to demonstrate constructor and this keyword.

Program Analysis: The program demonstrates a **constructor** for initializing object data and the this keyword for referring to the current object's variables.
A constructor is automatically called when an object is created.

Source Code:

```
class Student {
    int rollNo;
    String name;

    // Constructor
    Student(int rollNo, String name) {
        this.rollNo = rollNo;
        this.name = name;
    }

    void display() {
        System.out.println("Roll No: " + this.rollNo);
        System.out.println("Name: " + this.name);
    }

    public static void main(String[] args) {
        Student s = new Student(101, "Rahul");
        s.display();
    }
}
```

OUTPUT:

```
Roll No: 101
Name: Rahul
```

Practical No: 7

Program: Write a Java program to demonstrate static variable and static method.

Program Analysis: The program demonstrates a **static variable**, which is shared by all objects of a class, and a **static method**, which can be called without creating an object.
The static method displays the value of the static variable.

Source Code:

```
class StaticDemo
 {
   static int count = 10;   // Static variable

   static void display() {  // Static method
      System.out.println("Static variable value: " + count);
   }

   public static void main(String[] args) {
      StaticDemo.display();  // Calling static method
   }
}
```

OUTPUT:

```
Static variable value: 10
```

Practical No: 8

Program: Write a Java program to demonstrate inheritance.

Program Analysis: The program demonstrates **inheritance**, where a child class acquires the properties and methods of a parent class.Here, Dog inherits the eat() method from the Animal class.

Source Code:

```
class Animal
 {
   void eat()
 {
       System.out.println("Animal is eating");
   }
}
class Dog extends Animal
{
   void bark()
 {
       System.out.println("Dog is barking");
   }
   public static void main(String[] args)
{
       Dog d = new Dog();
       d.eat();   // Inherited method
       d.bark();  // Own method
   }
}
```

Output:

```
Animal is eating
Dog is barking
```

Practical No: 9

Program: Write a Java program to demonstrate method overloading and methog overriding.

Program Analysis: The program demonstrates **method overloading** by using the same method name with different parameters, and **method overriding** by redefining a parent class method in the child class.
Overloading is resolved at compile time, while overriding is resolved at runtime.

Source Code:

```
class Parent {
    // Method for overriding
    void show() {
        System.out.println("Parent class method");
    }
}
class Child extends Parent {
    @Override
    void show() {
        System.out.println("Child class method");
    }
    void add(int a, int b) {
        System.out.println("Sum = " + (a + b));
    }
    void add(int a, int b, int c) {
        System.out.println("Sum = " + (a + b + c));
    }
    public static void main(String[] args) {
        Child obj = new Child();
        obj.show();
        obj.add(10, 20);
        obj.add(10, 20, 30);
    }
}
```

OUTPUT:

```
Child class method
Sum = 30
Sum = 60
```

Practical No: 10

Program: Write a java program to create an interface shape with the getArea() method. Create three classes Rectangle,Circle,and Triangle that implements the shape interface. Implement the getArea() method for each of the three classes.

Program Analysis: The program creates a Shape interface with the getArea() method, which is implemented by Rectangle, Circle, and Triangle.
Each class provides its own implementation of getArea() to calculate the area of that particular shape.

Source Code:

```
interface Shape {
   double getArea();
}
class Rectangle implements Shape {
   double length = 10;
   double width = 5;

   public double getArea() {
      return length * width;
   }
}
class Circle implements Shape {
   double radius = 7;

   public double getArea() {
      return Math.PI * radius * radius;
   }
}
class Triangle implements Shape {
   double base = 8;
   double height = 6;

   public double getArea() {
      return 0.5 * base * height;
   }
}
```

```
class Main {
   public static void main(String[] args) {
      Shape r = new Rectangle();
      Shape c = new Circle();
      Shape t = new Triangle();

      System.out.println("Area of Rectangle = " + r.getArea());
      System.out.println("Area of Circle = " + c.getArea());
      System.out.println("Area of Triangle = " + t.getArea());
   }
}
```

OUTPUT:

```
Area of Rectangle = 50.0
Area of Circle = 153.93804002589985
Area of Triangle = 24.0
```

Practical No: 11

Program: Write a Java Program to demonstrate Abstract class and abstract method.

Program Analysis: The program demonstrates an **abstract class** Animal containing an abstract method sound() and a concrete method eat().
The Dog **class extends** Animal and implements sound(), demonstrating abstraction in Java.

Source Code:

```
abstract class Animal
 {
   // Abstract method
   abstract void sound();

   // Concrete method
   void eat() {
      System.out.println("Animal eats food.");
   }
}

// Subclass
class Dog extends Animal {
   // Providing implementation of abstract method
   void sound() {
      System.out.println("Dog barks.");
   }
}

public class Main {
   public static void main(String[] args) {
      Dog d = new Dog();

      d.sound();
      d.eat();
   }
}
```

OUTPUT:

```
Dog barks.
Animal eats food.
```

Practical No: 12

Program: Create a package College and add two classes Student and Staff to it.Add appropriate members to the classes.

Program Analysis: This Java program creates a package named College containing two classes: Student and Staff. Each class contains appropriate data members and methods to store and display information.

Source Code:

1. Student.java

```
package College;
public class Student {
    int rollNo;
    String name;
    String course;
    int age;
    public Student(int rollNo, String name, String course, int age) {
        this.rollNo = rollNo;
        this.name = name;
        this.course = course;
        this.age = age;
    }
    public void displayStudent() {
        System.out.println("Student Details:");
        System.out.println("Roll No: " + rollNo);
        System.out.println("Name: " + name);
        System.out.println("Course: " + course);
        System.out.println("Age: " + age);
    }
}
```

2. Staff.java

```
package College;
public class Staff {
    int staffId;
    String name;
```

```
    String department;
    String designation;

    public Staff(int staffId, String name, String department, String designation) {
        this.staffId = staffId;
        this.name = name;
        this.department = department;
        this.designation = designation;
    }

    public void displayStaff() {
        System.out.println("Staff Details:");
        System.out.println("Staff ID: " + staffId);
        System.out.println("Name: " + name);
        System.out.println("Department: " + department);
        System.out.println("Designation: " + designation);
    }
}
```

3. Main.java

```
import College.Student;
import College.Staff;
public class Main {
    public static void main(String[] args) {
        Student s = new Student(101, "Rahul", "BCA", 20);
        Staff st = new Staff(501, "Priya", "Computer Science", "Professor");
        s.displayStudent();
          System.out.println();
  st.displayStaff();
  }

}
```

OUTPUT:

```
Student Details:
Roll No: 101
Name: Rahul
Course: BCA
Age: 20

Staff Details:
Staff ID: 501
Name: Priya
Department: Computer Science
Designation: Professor
```

Practical No: 13

Program: Write a Java program that throws an exception and catch it using a try-catch block.

Program Analysis: The try block generates an ArithmeticException by dividing a number by zero.The catch block catches the exception and displays an appropriate message.

Source Code:

```
public class ExceptionDemo {

   public static void main(String[] args) {

      try {

         int a = 10 / 0;

      } catch (ArithmeticException e) {

         System.out.println("Exception caught: Cannot divide by zero");

      }

   }

}
```

OUTPUT:

```
Exception caught: Cannot divide by zero
```

Practical No: 14

Program: Write a Java program to create a thread using:
.Extending Thread class
.Implementing Runnable interface

Program Analysis:

1. **Extending Thread class**

```
class A extends Thread { public void run(){ System.out.println("Thread running"); } }
class Test { public static void main(String[] args){ new A().start(); } }
```

2. Implementing Runnable interface

```
class A implements Runnable { public void run(){ System.out.println("Thread running"); } }
class Test { public static void main(String[] args){ new Thread(new A()).start(); } }
```

Source Code:

```
// By Extending the Thread Class
class MyThread extends Thread {
   public void run() {
      System.out.println("Thread is running...");
   }
}
public class ThreadExample1 {
   public static void main(String[] args) {
      MyThread t = new MyThread();
      t.start();
   }
}
// By Implementing the Runnable Interface
class MyRunnable implements Runnable {
   public void run() {
      System.out.println("Thread is running...");
   }
}
public class ThreadExample2 {
   public static void main(String[] args) {
      MyRunnable r = new MyRunnable();
      Thread t = new Thread(r);
      t.start();
   }
}
```

OUTPUT:

```
Thread is running...
Thread is running...
```

Practical No: 15

Program: Write an Awt program to demonstrate FlowLayout, GridLayout, BorderLayout and CardLayout.

Program Analysis: This program demonstrates **FlowLayout, GridLayout, BorderLayout and CardLayout** using AWT components.Different buttons are arranged according to each layout manager.

Source Code:

```
import java.awt.*;
class LayoutDemo extends Frame {
   LayoutDemo() {
      setLayout(new BorderLayout());
      Panel f = new Panel(new FlowLayout());
      f.add(new Button("One"));
      f.add(new Button("Two"));

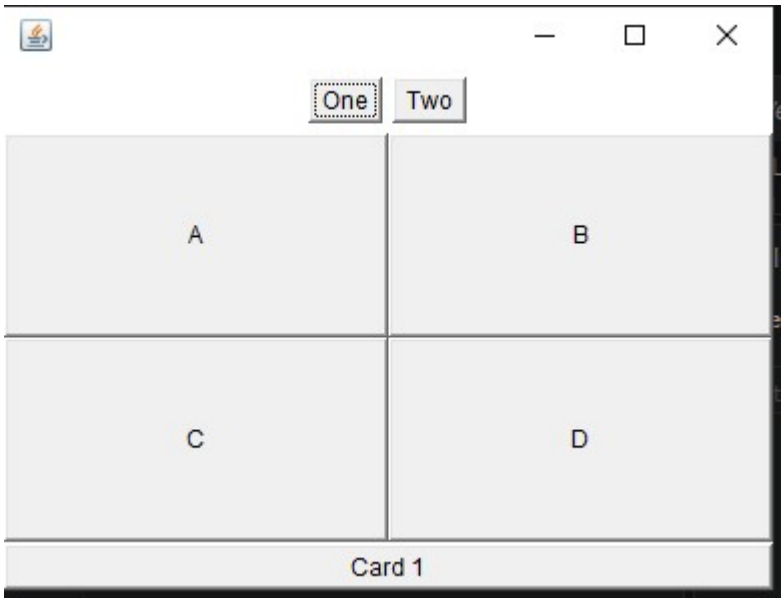
      Panel g = new Panel(new GridLayout(2, 2));
      g.add(new Button("A"));
      g.add(new Button("B"));
      g.add(new Button("C"));
      g.add(new Button("D"));

      Panel c = new Panel(new CardLayout());
      c.add(new Button("Card 1"));
      c.add(new Button("Card 2"));

      add(f, BorderLayout.NORTH);
      add(g, BorderLayout.CENTER);
      add(c, BorderLayout.SOUTH);

      setSize(400, 300);
      setVisible(true);
   }
   public static void main(String[] args) {
      new LayoutDemo();
   }
```

OUTPUT:



Practical No: 16

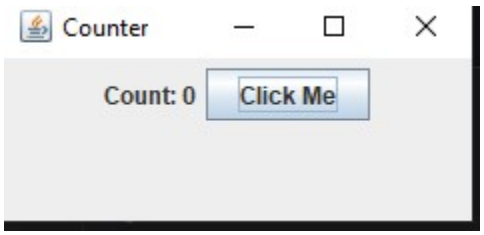
Program: Design a simple GUI with a Label and Button. Each time user clicks the button increase counter and display number of counts on the label.

Program Analysis: Create a Java GUI using **JFrame, JLabel, and JButton**, with a counter initialized to 0. Whenever the button is clicked, increase the counter by 1 and update the label.

Source Code:

```
import javax.swing.*;
public class CounterGUI {
  static int count = 0;
   public static void main(String[] args) {
      JFrame frame = new JFrame("Counter");
      JLabel label = new JLabel("Count: 0");
      JButton button = new JButton("Click Me");
      button.addActionListener(e -> {
         count++;
         label.setText("Count: " + count);
      });
      frame.setLayout(new java.awt.FlowLayout());
      frame.add(label);
      frame.add(button);
      frame.setSize(250, 120);
      frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
      frame.setVisible(true);
   }
}
```

OUTPUT:



Practical No: 17

Program: Develop a GUI application using AWT to create a simple calculator using:

. Label
. TextField
. Button
. Layout Managers

Program Analysis: Create an AWT calculator using Label, TextField, and Button components with GridLayout. Perform addition, subtraction, multiplication, and division when the corresponding button is clicked.

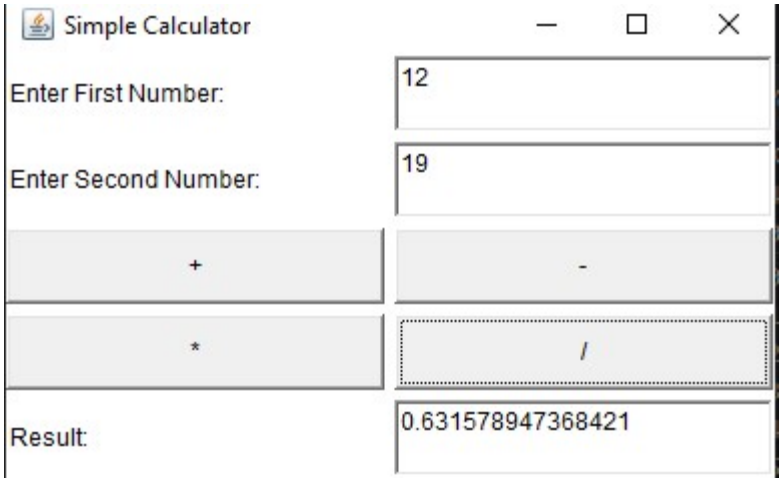
Source Code:

```
import java.awt.*;
import java.awt.event.*;
public class Calculator extends Frame implements ActionListener {
   Label l1, l2, l3;
   TextField t1, t2, t3;
   Button add, sub, mul, div;
   Calculator() {
      l1 = new Label("Enter First Number:");
      l2 = new Label("Enter Second Number:");
      l3 = new Label("Result:");
      t1 = new TextField();
      t2 = new TextField();
      t3 = new TextField();
      add = new Button("+");
      sub = new Button("-");
      mul = new Button("*");
      div = new Button("/");
      setLayout(new GridLayout(5, 2, 5, 5));
      add(l1);
      add(t1);
      add(l2);
      add(t2);
      add(add);
```

```
        add(sub);
        add(mul);
        add(div);
        add(l3);
        add(t3);
        add.addActionListener(this);
        sub.addActionListener(this);
        mul.addActionListener(this);
        div.addActionListener(this);
        setTitle("Simple Calculator");
        setSize(400, 250);
        setVisible(true);
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });
    }
    public void actionPerformed(ActionEvent e) {
        double a = Double.parseDouble(t1.getText());
        double b = Double.parseDouble(t2.getText());
        double result = 0;

        if (e.getSource() == add)
            result = a + b;
        else if (e.getSource() == sub)
            result = a - b;
        else if (e.getSource() == mul)
            result = a * b;
        else if (e.getSource() == div)
            result = a / b;
        t3.setText(String.valueOf(result));
    }
    public static void main(String[] args) {
        new Calculator();
    }
}
```

OUTPUT:


Simple Calculator
Enter First Number:
12
Enter Second Number:
19
+
-
*
/
Result:
0.631578947368421

Practical No: 18

Program: Design a simple GUI Temperature converter, using appropriate component.

Program Analysis: Create a Java AWT GUI using **Label, TextField, and Button** to enter temperature in Celsius. When the button is clicked, convert Celsius to Fahrenheit using F = (C × 9/5) + 32 and display the result.

Source Code:

```
import java.awt.*;
import java.awt.event.*;

public class TemperatureConverter extends Frame implements ActionListener {
   Label l1, l2;
   TextField t1, t2;
   Button b1;
   TemperatureConverter() {
      l1 = new Label("Enter Celsius:");
      l2 = new Label("Fahrenheit:");

      t1 = new TextField();
      t2 = new TextField();

      b1 = new Button("Convert");

      setLayout(new GridLayout(3, 2, 5, 5));

      add(l1);
      add(t1);
      add(l2);
      add(t2);
      add(b1);

      b1.addActionListener(this);

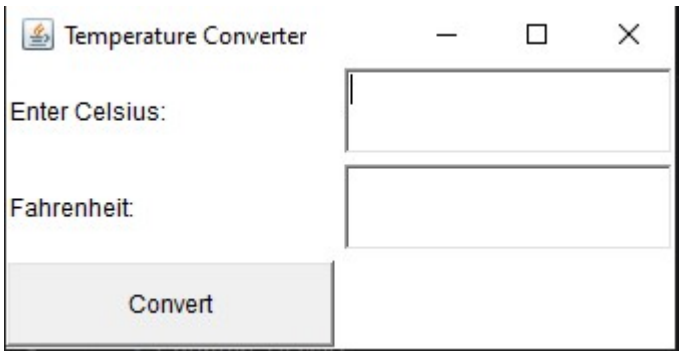
      setTitle("Temperature Converter");
      setSize(350, 180);
      setVisible(true);
```

```
        addWindowListener(new WindowAdapter() {
            public void windowClosing(WindowEvent e) {
                dispose();
            }
        });
    }

    public void actionPerformed(ActionEvent e) {
        double c = Double.parseDouble(t1.getText());
        double f = (c * 9 / 5) + 32;
        t2.setText(String.valueOf(f));
    }

    public static void main(String[] args) {
        new TemperatureConverter();
    }
}
```

OUTPUT:



Practical No: 19

Program: Write a java program using swing to create a frame having three text fields. Accept number in first textfield and display previous number in second textfield and next number in the third textfield.

Program Analysis: This program creates a Swing frame with three text fields and a button. It accepts a number and displays its previous number (n-1) and next number (n+1).

Source Code:

```
import javax.swing.*;

import java.awt.*;

import java.awt.event.*;

public class PreviousNext extends JFrame implements ActionListener {

   JTextField t1, t2, t3;

   JButton b;

   PreviousNext() {

      setTitle("Previous and Next Number");

      t1 = new JTextField(10);

      t2 = new JTextField(10);

      t3 = new JTextField(10);

      b = new JButton("Calculate");

      setLayout(new FlowLayout());

      add(new JLabel("Enter Number:"));

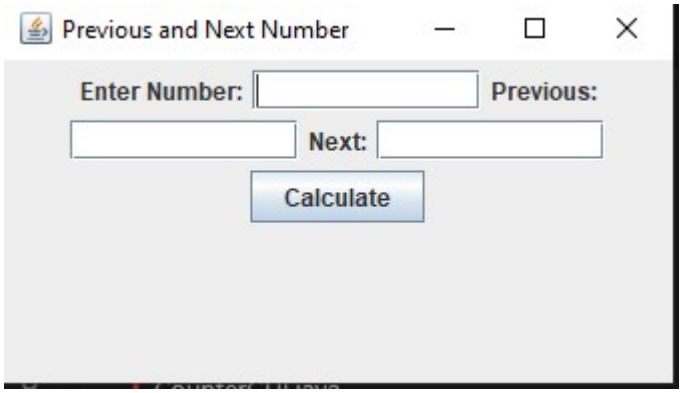
      add(t1);

      add(new JLabel("Previous:"));

      add(t2);
```

```
        add(new JLabel("Next:"));
        add(t3);
        add(b);
        b.addActionListener(this);
        setSize(350, 200);
        setVisible(true);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    }
     public void actionPerformed(ActionEvent e) {
        int n = Integer.parseInt(t1.getText());
        t2.setText(String.valueOf(n - 1));
        t3.setText(String.valueOf(n + 1));
    }
    public static void main(String[] args) {
        new PreviousNext();
    }
}
```

OUTPUT:



Practical No: 20

Program: Write a Java program that works as a simple calculator. Use a grid layout to arrange buttons for the digits and for the +, -, *, % operations. Add a text field to display the result. Handle any possible exceptions like divide by zero.

Program Analysis: This program creates a simple calculator using Java Swing components and GridLayout.It performs +, -, *, % operations and handles division/modulo by zero using exception handling.

Source Code:

```
import javax.swing.*;
import java.awt.*;
import java.awt.event.*;

public class Calculator extends JFrame implements ActionListener {

    JTextField display;
    double num1, num2;
    char operator;

    Calculator() {
        setTitle("Simple Calculator");
        setSize(300, 400);
        setLayout(new BorderLayout());

        display = new JTextField();
        add(display, BorderLayout.NORTH);

        JPanel panel = new JPanel();
        panel.setLayout(new GridLayout(4, 4, 5, 5));

        String buttons[] = {
            "7", "8", "9", "+",
            "4", "5", "6", "-",
            "1", "2", "3", "*",
            "0", "%", "=", "C"
        };
```

```
    for (String text : buttons) {
        JButton button = new JButton(text);
        button.addActionListener(this);
        panel.add(button);
    }

    add(panel, BorderLayout.CENTER);
    setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
    setVisible(true);
}

public void actionPerformed(ActionEvent e) {
    String command = e.getActionCommand();

    try {
        if (command.matches("[0-9]")) {
            display.setText(display.getText() + command);
        }
        else if (command.equals("C")) {
            display.setText("");
        }
        else if (command.equals("+") || command.equals("-") ||
                 command.equals("*") || command.equals("%")) {
            num1 = Double.parseDouble(display.getText());
            operator = command.charAt(0);
            display.setText("");
        }
        else if (command.equals("=")) {
            num2 = Double.parseDouble(display.getText());
            double result = 0;

            switch (operator) {
                case '+': result = num1 + num2; break;
                case '-': result = num1 - num2; break;
                case '*': result = num1 * num2; break;
                case '%':
                    if (num2 == 0)
                        throw new ArithmeticException("Cannot divide by zero");
                    result = num1 % num2;
```

```
                    break;
                }

                display.setText(String.valueOf(result));
            }
        }
        catch (ArithmeticException ex) {
            display.setText("Cannot divide by zero");
        }
        catch (Exception ex) {
            display.setText("Invalid Input");
        }
    }

    public static void main(String[] args) {
        new Calculator();
    }
}
```

OUTPUT:

